
Question 1: Building a Classifier

You have used Pandas for a small part of your Assignment 1. Now, we will see what we can do with it. We will use the same dataset (IMDB movie review dataset) that you used in Assignment 1.

a. Load the dataset using Pandas [1 mark]

Load the IMDB movie review dataset using Pandas, just like in Assignment 1.

b. Convert reviews using Bag-of-Words (CountVectorizer) [1 mark]

Use a **bag-of-words** model (known as `CountVectorizer` in scikit-learn) to convert the textual reviews into numeric feature vectors.

c. Split the dataset into training and testing [1 mark]

Split the dataset into two parts:

- **Training set (80%)**
- **Testing set (20%)**

You will train a machine learning model using the training data and evaluate its performance on the testing data.

d. Train two machine learning models [5 marks]

Train the following two models using the training data:

- **Logistic Regression**
- **Naive Bayes (MultinomialNB)**

Use scikit-learn to train both models.

e. Compare model performances [2 marks]

Compare the accuracies of the two models. Use any visualization techniques you like (e.g., bar graphs, charts).

- Indicate which model performs better.
 - Provide a brief explanation for the result.
-

Question 2: Building a Classifier (continued) [5 marks]

Repeat all tasks from **Question 1**, but this time use **TF-IDF Vectorizer** instead of **CountVectorizer**.

You should:

- Convert reviews using **TF-IDF Vectorizer**
- Train **Logistic Regression** and **Naive Bayes** models on the new TF-IDF features
- Compare their performances just like before

Finally, compare:

- **Bag-of-Words vs TF-IDF**
 - Which model gives better results overall? Explain why you think that is the case.
-

Question 3: Training a Deep Learning Model [5 marks]

In this question, you will train a shallow (single hidden layer) Recurrent Neural Network (RNN) to classify movie reviews.

Instructions:

- Follow the tutorial given in the course materials.
- Your model should have **three layers**:
 1. **Embedding Layer**
 - Output shape: 100
 - Converts each word into a 100-dimensional vector
 - Load **pre-trained GloVe embeddings** into this layer
 2. **Dense Layer**
 - Output shape: 10
 3. **Output Layer**
 - Output shape: 1 (binary classification)

Do **not** use any pooling or additional layers.

Training:

- Run your model for **50 epochs**
- Adjust hyperparameters (e.g., batch size) if needed

Evaluation:

- Compare the RNN model's performance with the classical models (Logistic Regression and Naive Bayes).
 - Summarize how well the RNN performs and provide insights based on the results.
-